7.3 Requirements Gathering 110
7.3.1 Collaborative Requirements Gathering 110
7.3.2 Usage Scenarios 113
7.3.3 Elicitation Work Products 114
7.4 Developing Use Cases 114
7.5 Building the Analysis Model 118
7.5.1 Elements of the Analysis Model 119
7.5.2 Analysis Patterns 122
7.6 Negotiating Requirements 122
7.7 Requirements Monitoring 123
7.8 Validating Requirements 123
7.9 Summary 124

CHAPTER 8 REQUIREMENTS MODELING—A RECOMMENDED APPROACH 126

8.1 Requirements Analysis 127
8.1.1 Overall Objectives and Philosophy 128
8.1.2 Analysis Rules of Thumb 128
8.1.3 Requirements Modeling Principles 129
8.2 Scenario-Based Modeling 130
8.2.1 Actors and User Profiles 131
8.2.2 Creating Use Cases 131
8.2.3 Documenting Use Cases 135
8.3 Class-Based Modeling 137
8.3.1 Identifying Analysis Classes 137
8.3.2 Defining Attributes and Operations 140
8.3.3 UML Class Models 141
8.3.4 Class-Responsibility-Collaborator Modeling 144
8.4 Functional Modeling 146
8.4.1 A Procedural View 146
8.4.2 UML Sequence Diagrams 148
8.5 Behavioral Modeling 149
8.5.1 Identifying Events with the Use Case 149
8.5.2 UML State Diagrams 150
8.5.3 UML Activity Diagrams 151
8.6 Summary 154

CHAPTER 9 DESIGN CONCEPTS 156

9.1 Design Within the Context of Software Engineering 157
9.2 The Design Process 159
9.2.1 Software Quality Guidelines and Attributes 160
9.2.2 The Evolution of Software Design 161
9.3 Design Concepts 163
9.3.1 Abstraction 163
9.3.2 Architecture 163
9.3.3 Patterns 164
9.3.4 Separation of Concerns 165
9.3.5 Modularity 165
9.3.6 Information Hiding 166
9.3.7 Functional Independence 167
9.3.8 Stepwise Refinement 167
9.3.9 Refactoring 168
9.3.10 Design Classes 169
9.4 The Design Model 171
9.4.1 Design Modeling Principles 173
9.4.2 Data Design Elements 174
9.4.3 Architectural Design Elements 175
9.4.4 Interface Design Elements 175
9.4.5 Component-Level Design Elements 176
9.4.6 Deployment-Level Design Elements 177
9.5 Summary 178

CHAPTER 10 ARCHITECTURAL DESIGN— A RECOMMENDED APPROACH 181

10.1 Software Architecture 182
10.1.1 What Is Architecture? 182
10.1.2 Why Is Architecture Important? 183
10.1.3 Architectural Descriptions 183
10.1.4 Architectural Decisions 184
10.2 Agility and Architecture 185
10.3 Architectural Styles 186
10.3.1 A Brief Taxonomy of Architectural Styles 187
10.3.2 Architectural Patterns 192
10.3.3 Organization and Refinement 193

10.4 Architectural Considerations 193
10.5 Architectural Decisions 195
10.6 Architectural Design 196
10.6.1 Representing the System in Context 196
10.6.2 Defining Archetypes 197
10.6.3 Refining the Architecture into Components 198
10.6.4 Describing Instantiations of the System 200
10.7 Assessing Alternative Architectural Designs 201
10.7.1 Architectural Reviews 202
10.7.2 Pattern-Based Architecture Review 203
10.7.3 Architecture Conformance Checking 204
10.8 Summary 204

CHAPTER 11 COMPONENT-LEVEL DESIGN 206

11.1 What Is a Component? 207
11.1.1 An Object-Oriented View 207
11.1.2 The Traditional View 209
11.1.3 A Process-Related View 211
11.2 Designing Class-Based Components 212
11.2.1 Basic Design Principles 212
11.2.2 Component-Level Design Guidelines 215
11.2.3 Cohesion 216
11.2.4 Coupling 218
11.3 Conducting Component-Level Design 219
11.4 Specialized Component-Level Design 225
11.4.1 Component-Level Design for WebApps 226
11.4.2 Component-Level Design for Mobile Apps 226
11.4.3 Designing Traditional Components 227
11.4.4 Component-Based Development 228
11.5 Component Refactoring 230
11.6 Summary 231

CHAPTER 12 USER EXPERIENCE DESIGN 233

12.1 User Experience Design Elements 234
12.1.1 Information Architecture 235
12.1.2 User Interaction Design 236
12.1.3 Usability Engineering 236
12.1.4 Visual Design 237

12.2 The Golden Rules 238
12.2.1 Place the User in Control 238
12.2.2 Reduce the User's Memory Load 239
12.2.3 Make the Interface Consistent 240
12.3 User Interface Analysis and Design 241
12.3.1 Interface Analysis and Design Models 241
12.3.2 The Process 242
12.4 User Experience Analysis 243
12.4.1 User Research 244
12.4.2 User Modeling 245
12.4.3 Task Analysis 247
12.4.4 Work Environment Analysis 248
12.5 User Experience Design 249
12.6 User Interface Design 250
12.6.1 Applying Interface Design Steps 251
12.6.2 User Interface Design Patterns 252
12.7 Design Evaluation 253
12.7.1 Prototype Review 253
12.7.2 User Testing 255
12.8 Usability and Accessibility 255
12.8.1 Usability Guidelines 257
12.8.2 Accessibility Guidelines 259
12.9 Conventional Software UX and Mobility 261
12.10 Summary 261

CHAPTER 13 DESIGN FOR MOBILITY 264

13.1 The Challenges 265
13.1.1 Development Considerations 265
13.1.2 Technical Considerations 266
13.2 Mobile Development Life Cycle 268
13.2.1 User Interface Design 270
13.2.2 Lessons Learned 271
13.3 Mobile Architectures 273
13.4 Context-Aware Apps 274
13.5 Web Design Pyramid 275
13.5.1 WebApp Interface Design 275
13.5.2 Aesthetic Design 277
13.5.3 Content Design 277
13.5.4 Architecture Design 278
13.5.5 Navigation Design 280

13.6 Component-Level Design 282
13.7 Mobility and Design Quality 282
13.8 Mobility Design Best Practices 285
13.9 Summary 287

CHAPTER 14 PATTERN-BASED DESIGN 289

14.1 Design Patterns 290
14.1.1 Kinds of Patterns 291
14.1.2 Frameworks 293
14.1.3 Describing a Pattern 293
14.1.4 Machine Learning and Pattern Discovery 294
14.2 Pattern-Based Software Design 295
14.2.1 Pattern-Based Design in Context 295
14.2.2 Thinking in Patterns 296
14,2.3 Design Tasks 297
14.2.4 Building a Pattern-Organizing Table 298
14.2.5 Common Design Mistakes 298
14.3 Architectural Patterns 299
14.4 Component-Level Design Patterns 300
14.5 Anti-Patterns 302
14.6 User Interface Design Patterns 304
14.7 Mobility Design Patterns 305
14.8 Summary 306

PART THREE QUALITY AND SECURITY 309

CHAPTER 15 QUALITY CONCEPTS 310

15.1 What Is Quality? 311
15.2 Software Quality 312
15.2.1 Quality Factors 312
15.2.2 Qualitative Quality Assessment 314
15.2.3 Quantitative Quality Assessment 315
15.3 The Software Quality Dilemma 315
15.3.1 “Good Enough” Software 316
15.3.2 The Cost of Quality 317
15.3.3 Risks 319
15.3.4 Negligence and Liability 320

15.3.5 Quality and Security 320
15.3.6 The Impact of Management Actions 321
15.4 Achieving Software Quality 321
15.4.1 Software Engineering Methods 322
15.4.2 Project Management Techniques 322
15.4.3 Machine Learning and Defect Prediction 322
15.4.4 Quality Control 322
15.4.5 Quality Assurance 323
15.5 Summary 323

CHAPTER 16 REVIEWS—A RECOMMENDED APPROACH 325

16.1 Cost Impact of Software Defects 326
16.2 Defect Amplification and Removal 327
16.3 Review Metrics and Their Use 327
16.4 Criteria for Types of Reviews 330
16.5 Informal Reviews 331
16.6 Formal Technical Reviews 332
16.6.1 The Review Meeting 332
16.6.2 Review Reporting and Record Keeping 333
16.6.3 Review Guidelines 334
16.7 Postmortem Evaluations 336
16.8 Agile Reviews 336
16.9 Summary 337

CHAPTER 17 SOFTWARE QUALITY ASSURANCE 339

17.1 Background Issues 341
17.2 Elements of Software Quality Assurance 341
17.3 SQA Processes and Product Characteristics 343
17.4 SQA Tasks, Goals, and Metrics 343
17.4.1 SQA Tasks 343
17.4.2 Goals, Attributes, and Metrics 345
17.5 Formal Approaches to SQA 347
17.6 Statistical Software Quality Assurance 347